



INE-PYTHON

LECTURE - 10 PYTHON LIBRARIES AND MODULES

What You Will Learn

- Introduction
- Standard Library
- Third-Party Libraries

PYTHON LIBRARIES AND MODULES



Introduction

INTRODUCTION TO MODULES AND LIBRARIES

ในภาษาไพธอน โมดูลและไลบรารีมีบทบาทสำคัญในการจัดระเบียบและนำโค้ดกลับมาใช้ใหม่ พวกมันช่วยแบ่งโปรแกรมที่ซับซ้อนให้กลายเป็นส่วนย่อยที่จัดการได้ง่ายขึ้น ทำให้การพัฒนามีประสิทธิภาพและเป็นระบบมากยิ่งขึ้น ไลบรารีมาตรฐานของ Python มีคลังโค้ดสำเร็จรูปมากมายสำหรับการทำงานหลากหลายรูปแบบ และยังสามารถขยายขีดความสามารถเพิ่มเติมได้ด้วยไลบรารีของบุคคลที่สาม การเข้าใจวิธีใช้โมดูลและไลบรารีจึงเป็นสิ่งจำเป็นในการเขียนโปรแกรมที่สะอาด ดูแลรักษาง่าย และมีประสิทธิภาพ

การเรียนรู้เกี่ยวกับโมดูลและไลบรารีสามารถช่วยเพิ่มทักษะและประสิทธิภาพในการเขียนโปรแกรมได้อย่างมาก มันช่วยให้สามารถจัดระเบียบโค้ดได้ดีขึ้น และมอบเครื่องมือที่ทรงพลังในการแก้ไขงานที่ซับซ้อน ทำให้การเขียนโปรแกรมเป็นเรื่องที่สนุกและมีประสิทธิภาพยิ่งขึ้น

WHAT IS A MODULE?

แนวคิดสำคัญของโมดูล:

- คำนิยาม: ไฟล์ที่มีโค้ด Python เช่น ฟังก์ชัน คลาส ตัวแปร
- การจัดระเบียบ: ช่วยจัดโค้ดให้อยู่ในส่วนที่จัดการได้ง่าย
- การนำกลับมาใช้: ใช้งานโค้ดซ้ำได้ในหลายโปรแกรม
- การนำเข้า: ใช้คำสั่ง `import` เพื่อนำเข้าโมดูล
- เนมสเปซ: โมดูลสร้างเนมสเปซแยกต่างหากเพื่อหลีกเลี่ยงการชนกันของชื่อ

WHAT IS A LIBRARY?

ไลบรารีคือชุดของโมดูลที่เกี่ยวข้องกันซึ่งให้บริการเฉพาะด้าน ไลบรารีมาตรฐานของ Python เป็นตัวอย่างที่ดี ซึ่งมีชุดของโมดูลที่มาพร้อมกับภาษา ครอบคลุมฟังก์ชันหลากหลาย ตั้งแต่การคำนวณพื้นฐานและการจัดการสตริง ไปจนถึงงานที่ซับซ้อนกว่า เช่น การพัฒนาเว็บและการวิเคราะห์ข้อมูล ไลบรารีช่วยประหยัดเวลาและแรงงานโดยให้โค้ดที่เขียนและทดสอบไว้ล่วงหน้า ซึ่งสามารถนำมาใช้ในการพัฒนาโปรแกรมโดยไม่ต้องเริ่มต้นใหม่ทุกครั้ง

นอกจากไลบรารีมาตรฐานแล้ว ยังมีไลบรารีจากบุคคลที่สามอีกจำนวนมาก ซึ่งพัฒนาโดยนักพัฒนาอื่น ๆ และสามารถติดตั้งเพิ่มเติมได้ง่ายเพื่อขยายขีดความสามารถของ Python ยกตัวอย่างเช่น NumPy และ Pandas ที่ได้รับความนิยมในงานคำนวณเชิงวิทยาศาสตร์และการวิเคราะห์ข้อมูล หรือ Flask และ Django ที่ใช้ในการพัฒนาเว็บ

WHAT IS A LIBRARY?

แนวคิดสำคัญของไลบรารี:

- คำนิยาม: ชุดของโมดูลที่เกี่ยวข้องกัน
- ไลบรารีมาตรฐาน: มาพร้อมกับ Python ให้ฟังก์ชันหลากหลาย
- ไลบรารีจากบุคคลที่สาม: พัฒนาเพิ่มเติมโดยชุมชน สามารถติดตั้งผ่าน pip
- การนำโค้ดกลับมาใช้: มีโค้ดที่เขียนและทดสอบไว้แล้วสำหรับงานทั่วไป
- การขยายขีดความสามารถ: ช่วยขยายขอบเขตการใช้งานของ Python
- ตัวอย่าง: NumPy สำหรับการคำนวณเชิงตัวเลข, Pandas สำหรับการวิเคราะห์ข้อมูล, Flask สำหรับการพัฒนาเว็บ

PYTHON LIBRARIES AND MODULES



Commonly Used Modules in the Standard Library

OS MODULE

```
1 import os
2
3 print(os.name)      # Output: posix (on Unix-like systems), nt (on Windows)
4 print(os.getcwd()) # Output: Current working directory
5 os.mkdir("new_directory") # Create a new directory
```

Listing 10.5: os Module

```
posix
/Users/anirachmingkhan/Code/INE-Python
```


DATETIME MODULE

```
1 import datetime
2
3 now = datetime.datetime.now()
4 print(now)    # Output: Current date and time
5
6 date = datetime.date(2024, 1, 1)
7 print(date)   # Output: 2024-01-01
```

Listing 10.7: Example of datetime Module

```
2024-09-16 18:20:48.453765
2024-01-01
```

JSON MODULE

```
1 import json
2
3 data = {"name": "Alice", "age": 25}
4 json_str = json.dumps(data)
5 print(json_str)    # Output: JSON string
6
7 parsed_data = json.loads(json_str)
8 print(parsed_data)  # Output: Python dictionary
9 print(parsed_data["name"])  # Output: Alice
10 print(parsed_data["age"])  # Output: 25
```

Listing 10.9: Example of json Module

```
{"name": "Alice", "age": 25}
{'name': 'Alice', 'age': 25}
Alice
25
```

PYTHON LIBRARIES AND MODULES



Overview of Third-Party Libraries

INSTALLING THIRD-PARTY LIBRARIES

ไลบรารีจากบุคคลที่สามช่วยขยายความสามารถของภาษา Python ให้เหนือกว่าไลบรารีมาตรฐาน โดยมอบเครื่องมือและฟังก์ชันเฉพาะทางสำหรับการใช้งานที่หลากหลาย ไลบรารีเหล่านี้สามารถหาได้จาก Python Package Index (PyPI) ซึ่งเป็นแหล่งรวมซอฟต์แวร์ Python ที่ครอบคลุม นักพัฒนาสามารถค้นหา ดาวน์โหลด และติดตั้งไลบรารีเหล่านี้ได้อย่างง่ายดายเพื่อเพิ่มศักยภาพให้กับโปรเจกต์ของตน

เครื่องมือหลักสำหรับติดตั้งไลบรารีจากบุคคลที่สามคือ `pip` ซึ่งเป็นตัวจัดการแพ็คเกจของ Python โดยสามารถใช้คำสั่งง่าย ๆ เช่น `pip install` เพื่อติดตั้งไลบรารีใหม่เข้าสู่สภาพแวดล้อมของ Python ได้อย่างรวดเร็ว ความสะดวกนี้ช่วยส่งเสริมให้เกิดการพัฒนาแอปพลิเคชันที่แข็งแกร่งได้อย่างรวดเร็ว เพราะสามารถนำประโยชน์จากงานพัฒนาของชุมชน Python มาใช้ได้ ไม่ว่าจะเป็นการคำนวณเชิงตัวเลข การวิเคราะห์ข้อมูล การพัฒนาเว็บ หรือการเรียนรู้ของเครื่อง ก็มีไลบรารีให้เลือกใช้หลากหลาย ส่งเสริมให้เกิดนวัตกรรมและประสิทธิภาพในการพัฒนาโปรแกรม

INSTALLING THIRD-PARTY LIBRARIES

คุณสามารถติดตั้งไลบรารีจากบุคคลที่สามได้โดยใช้คำสั่ง `pip install` คำสั่งนี้จะช่วยให้คุณสามารถดาวน์โหลดและติดตั้งไลบรารีจาก PyPI ได้อย่างง่ายดาย เพียงเปิดโปรแกรมบรรทัดคำสั่ง (Command Line Interface) และพิมพ์ `pip install library_name` เพื่อเพิ่มไลบรารีที่ต้องการเข้าสู่สภาพแวดล้อมของ Python

```
1 pip install requests
```

NUMPY MODULE

NumPy เป็นไลบรารีที่ทรงพลังสำหรับการคำนวณเชิงตัวเลขในภาษา Python ซึ่งถูกใช้กันอย่างแพร่หลายในด้านการคำนวณทางวิทยาศาสตร์ การวิเคราะห์ข้อมูล และการเรียนรู้ของเครื่อง ไลบรารีนี้ให้การสนับสนุนอย่างเต็มที่สำหรับอาร์เรย์และเมทริกซ์ ซึ่งเป็นโครงสร้างข้อมูลพื้นฐานที่จำเป็นสำหรับการคำนวณเชิงตัวเลขอย่างมีประสิทธิภาพ

NumPy ประกอบด้วยฟังก์ชันทางคณิตศาสตร์ที่หลากหลาย เช่น พีชคณิตเชิงเส้น การวิเคราะห์ทางสถิติ และการแปลงฟูริเยร์ ออบเจกต์อาร์เรย์ของ NumPy ได้รับการปรับแต่งให้มีประสิทธิภาพสูง จึงสามารถดำเนินการคำนวณได้อย่างรวดเร็ว นอกจากนี้ยังสามารถรวมการใช้งานร่วมกับไลบรารีวิทยาศาสตร์อื่น ๆ เช่น SciPy และ Pandas เพื่อสร้างเวิร์กโฟลว์สำหรับการจัดการและวิเคราะห์ข้อมูลที่ซับซ้อนได้อย่างราบรื่น ด้วยความสามารถในการจัดการข้อมูลขนาดใหญ่และการดำเนินการทางคณิตศาสตร์ขั้นสูง NumPy จึงเป็นเครื่องมือพื้นฐานที่นักพัฒนาและนักวิจัยในสายงานที่ต้องการการประมวลผลเชิงตัวเลขไม่ควรขาด

NUMPY MODULE

```
1 import numpy as np
2
3 # Create a 3x3 array of random integers between 1 and 10
4 random_matrix = np.random.randint(1, 11, size=(3, 3))
5 print("Random 3x3 Matrix:\n", random_matrix)
6
7 # Find the sum of all elements in the matrix
8 matrix_sum = np.sum(random_matrix)
9 print(f"\nSum of all elements: {matrix_sum}")
10
11 # Find the mean of the matrix
12 matrix_mean = np.mean(random_matrix)
13 print(f"\nMean of the matrix: {matrix_mean:2.2f}")
14
15 # Transpose the matrix
16 transposed_matrix = np.transpose(random_matrix)
17 print("\nTransposed Matrix:\n", transposed_matrix)
```

Random 3x3 Matrix:

```
[[ 9  9 10]
 [ 3  2  9]
 [ 6  3  2]]
```

Sum of all elements: 53

Mean of the matrix: 5.89

Transposed Matrix:

```
[[ 9  3  6]
 [ 9  2  3]
 [10  9  2]]
```

Listing 10.10: Example of NumPy

PANDAS MODULE

Pandas เป็นไลบรารีที่มีความยืดหยุ่นและทรงพลังสำหรับการจัดการและวิเคราะห์ข้อมูลในภาษา Python ซึ่งได้รับความนิยมในสาขาวิทยาศาสตร์ข้อมูล การเงิน เศรษฐศาสตร์ และสาขาอื่น ๆ ที่เกี่ยวข้องกับชุดข้อมูลขนาดใหญ่ โดยไลบรารีนี้ให้โครงสร้างข้อมูลที่มีประสิทธิภาพ เช่น **Series** และ **DataFrame** ซึ่งเหมาะสำหรับการจัดการข้อมูลแบบตาราง

DataFrame โดยเฉพาะ เป็นโครงสร้างข้อมูลแบบสองมิติ ที่สามารถปรับขนาดได้ และเก็บข้อมูลหลากหลายประเภท พร้อมทั้งมีป้ายกำกับแถวและคอลัมน์ คล้ายกับสเปรดชีตหรือฐานข้อมูล SQL Pandas มีฟังก์ชันครบถ้วนสำหรับการจัดการข้อมูล เช่น การอ่าน/เขียนจากหลายรูปแบบไฟล์ (CSV, Excel, SQL) การทำความสะอาดข้อมูล การกรอง การรวมข้อมูล การจัดกลุ่ม และการปรับโครงสร้างข้อมูล

PANDAS MODULE

```
1 import pandas as pd
2
3 # Create a DataFrame from a dictionary
4 data = {'Name': ['Alice', 'Bob', 'Charlie'],
5         'Age': [25, 30, 35],
6         'City': ['New York', 'Los Angeles', 'Chicago']}
7
8 df = pd.DataFrame(data)
9 print("DataFrame:\n", df)
10
11 # Calculate the average age
12 average_age = df['Age'].mean()
13 print("\nAverage Age:", average_age)
14
15 # Filter rows where Age is greater than 28
16 filtered_df = df[df['Age'] > 28]
17 print("\nFiltered DataFrame (Age > 28):\n", filtered_df)
18
19 # Add a new column for salary
20 df['Salary'] = [50000, 60000, 70000]
21 print("\nDataFrame with Salary column:\n", df)
```

DataFrame:

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

Average Age: 30.0

Filtered DataFrame (Age > 28):

	Name	Age	City
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

DataFrame with Salary column:

	Name	Age	City	Salary
0	Alice	25	New York	50000
1	Bob	30	Los Angeles	60000
2	Charlie	35	Chicago	70000

Listing 10.11: Example of Pandas

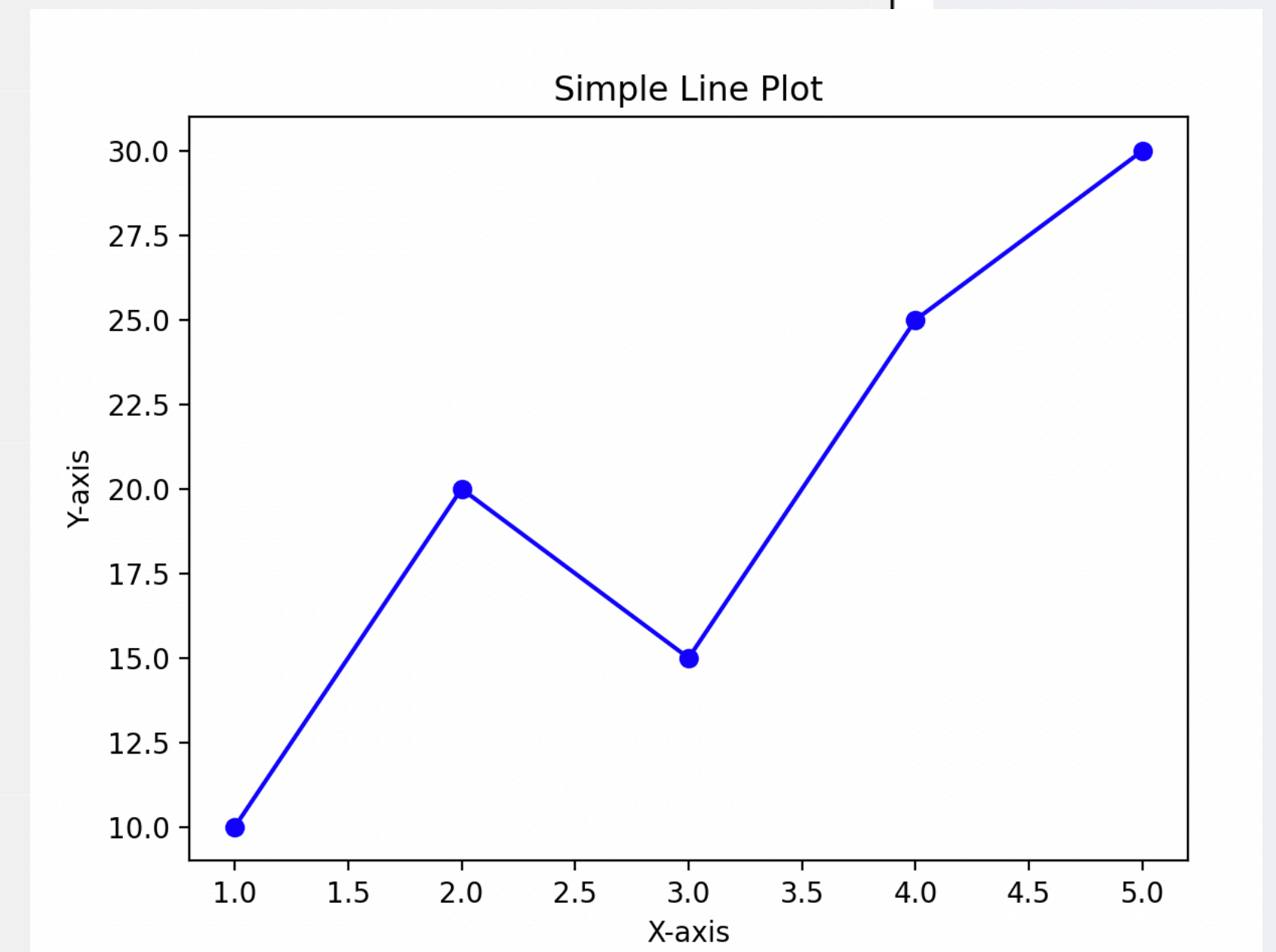
MATPLOTLIB MODULE

Matplotlib เป็นไลบรารีที่มีความยืดหยุ่นสูงสำหรับการสร้างภาพข้อมูลทั้งแบบคงที่ แบบเคลื่อนไหว และแบบอินเทอร์แอคทีฟในภาษา Python ได้รับความนิยมอย่างกว้างขวางในด้านวิทยาศาสตร์ข้อมูล วิศวกรรม การเงิน และสาขาอื่น ๆ ที่ต้องการการแสดงผลข้อมูลที่ชัดเจนและเข้าใจง่าย

ด้วย Matplotlib คุณสามารถสร้างกราฟและแผนภูมิได้หลากหลายรูปแบบ เช่น กราฟเส้น แผนภูมิแท่ง ฮิสโตแกรม กราฟกระจาย และกราฟสามมิติ การออกแบบของไลบรารีนี้มีลักษณะคล้ายกับ MATLAB จึงเหมาะสำหรับผู้ใช้ที่คุ้นเคยกับ MATLAB อยู่แล้ว Matplotlib ให้คุณควบคุมองค์ประกอบของกราฟได้ละเอียด เช่น สี ป้ายชื่อ สไตล์เส้น และฟอนต์ ทำให้สามารถปรับแต่งกราฟให้เหมาะสมกับความต้องการเฉพาะได้

MATPLOTLIB MODULE

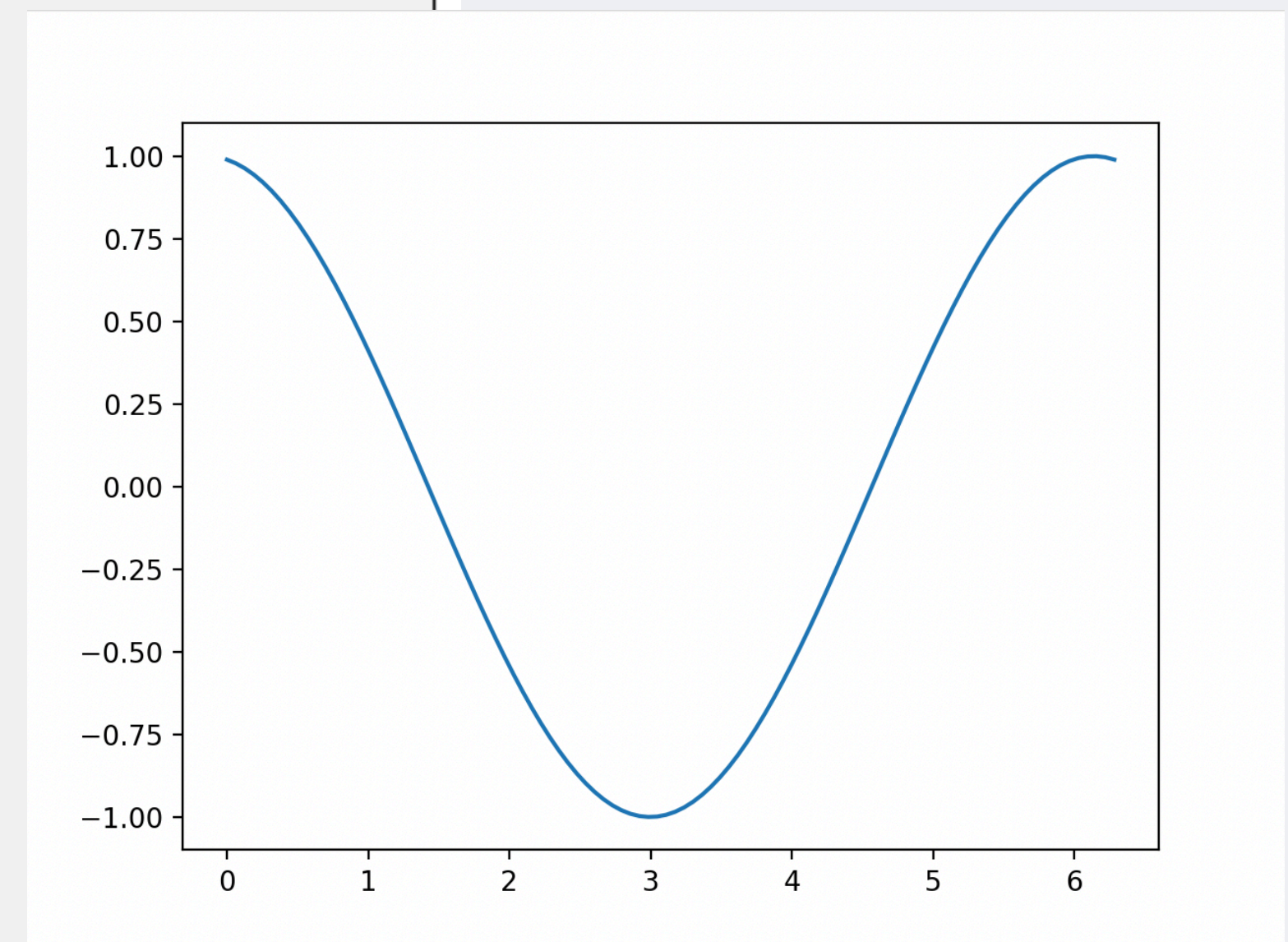
```
1 import matplotlib.pyplot as plt
2
3 # Data for plotting
4 x = [1, 2, 3, 4, 5]
5 y = [10, 20, 15, 25, 30]
6
7 # Create a line plot
8 plt.plot(x, y, marker='o', linestyle='-', color='b')
9
10 # Add labels and title
11 plt.xlabel('X-axis')
12 plt.ylabel('Y-axis')
13 plt.title('Simple Line Plot')
14
15 # Display the plot
16 plt.show()
```



Listing 10.12: Example of Matplotlib

MATPLOTLIB MODULE

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import matplotlib.animation as animation
4
5 # Create a figure and an axis
6 fig, ax = plt.subplots()
7
8 # Generate initial data
9 x = np.linspace(0, 2 * np.pi, 100)
10 y = np.sin(x)
11
12 # Create a line object that will be updated during the animation
13 line, = ax.plot(x, y)
14
15 # Function to update the plot for each frame
16 def update(frame):
17     line.set_ydata(np.sin(x + frame / 10.0)) # Shift the sine wave
18     return line,
19
20 # Create an animation object
21 ani = animation.FuncAnimation(fig, update, frames=100, interval=50, blit=True)
22
23 # Display the animation
24 plt.show()
```



Listing 10.13: Example of Animation Matplotlib

REQUESTS MODULE

Requests เป็นไลบรารียอดนิยมที่ใช้ทำงานง่ายสำหรับการส่งคำขอ HTTP ในภาษา Python มันช่วยให้การโต้ตอบกับเว็บเซอร์วิสและ API เป็นเรื่องง่าย โดยนักพัฒนาสามารถส่งคำขอและจัดการการตอบกลับได้อย่างไม่ซับซ้อน

ด้วยอินเทอร์เฟซที่เรียบง่ายของ Requests คุณสามารถใช้เมธอด HTTP ได้หลากหลาย เช่น **GET**, **POST**, **PUT**, **DELETE** และอื่น ๆ นอกจากนี้ยังสามารถจัดการเรื่องที่ซับซ้อน เช่น การยืนยันตัวตน เซสชัน คุกกี้ และการเปลี่ยนเส้นทาง ได้อย่างมีประสิทธิภาพ

Requests เหมาะสำหรับงานเชื่อมต่อ API, ดึงข้อมูลจากเว็บไซต์ (web scraping) และการสื่อสารผ่านเครือข่าย โดยมีไวยากรณ์ที่เข้าใจง่ายและเอกสารประกอบที่ครอบคลุม จึงถือเป็นเครื่องมือสำคัญสำหรับนักพัฒนาที่ต้องติดต่อกับเว็บผ่านแอปพลิเคชัน Python

REQUESTS MODULE

```
1 import requests
2
3 # Make a GET request to a public API (GitHub API)
4 response = requests.get('https://api.github.com/users/octocat')
5
6 # Check if the request was successful
7 if response.status_code == 200:
8     user_data = response.json()
9
10    # Display some information from the response
11    print(f"Username: {user_data['login']}")
12    print(f"Name: {user_data['name']}")
13    print(f"Bio: {user_data['bio']}")
14    print(f"Public Repos: {user_data['public_repos']}")
15    print(f"Followers: {user_data['followers']}")
16    print(f"Following: {user_data['following']}")
17 else:
18    print("Failed to retrieve data.")
```

```
> python Req.py
{'login': 'octocat', 'id': 583231, 'node_id': 'MDQ6VXNlcjU4MzIzMQ==', 'avatar_url': 'https://avatars.githubusercontent.com/u/583231?v=4', 'gravatar_id': '', 'url': 'https://api.github.com/users/octocat', 'html_url': 'https://github.com/octocat', 'followers_url': 'https://api.github.com/users/octocat/followers', 'following_url': 'https://api.github.com/users/octocat/following{/other_user}', 'gists_url': 'https://api.github.com/users/octocat/gists{/gist_id}', 'starred_url': 'https://api.github.com/users/octocat/starring{/owner}/{/repo}', 'subscriptions_url': 'https://api.github.com/users/octocat/subscriptions', 'organizations_url': 'https://api.github.com/users/octocat/orgs', 'repos_url': 'https://api.github.com/users/octocat/repos', 'events_url': 'https://api.github.com/users/octocat/events{/privacy}', 'received_events_url': 'https://api.github.com/users/octocat/received_events', 'type': 'User', 'site_admin': False, 'name': 'The Octocat', 'company': '@github', 'blog': 'https://github.blog', 'location': 'San Francisco', 'email': None, 'hireable': None, 'bio': None, 'twitter_username': None, 'public_repos': 8, 'public_gists': 8, 'followers': 14946, 'following': 9, 'created_at': '2011-01-25T18:44:36Z', 'updated_at': '2024-08-22T11:25:04Z'}
Username: octocat
Name: The Octocat
Bio: None
Public Repos: 8
Followers: 14946
Following: 9
```

Listing 10.14: Example of Requests

FLASK MODULE

Flask เป็นไมโครเฟรมเวิร์กที่เบาและยืดหยุ่นสำหรับการพัฒนาเว็บแอปพลิเคชันด้วยภาษา Python ออกแบบมาให้ใช้งานง่าย เหมาะทั้งสำหรับผู้เริ่มต้นและนักพัฒนาที่มีประสบการณ์ โดยมีเครื่องมือพื้นฐานในการสร้างเว็บ เช่น การกำหนดเส้นทาง การจัดการคำขอ และการแสดงผลด้วยเทมเพลต

Flask เน้นความเรียบง่าย โดยไม่กำหนดโครงสร้างตายตัวให้กับแอปพลิเคชัน ผู้พัฒนาสามารถจัดโครงสร้างแอปตามที่ต้องการได้อย่างอิสระ นอกจากนี้ยังสามารถเพิ่มฟีเจอร์เพิ่มเติมได้ผ่านส่วนขยาย

Flask ทำงานบนมาตรฐาน WSGI (Web Server Gateway Interface) ซึ่งทำให้สามารถใช้งานร่วมกับเว็บเซิร์ฟเวอร์ต่าง ๆ ได้ง่าย และสามารถรวมเข้ากับไลบรารีอื่นของ Python ได้อย่างดี เช่น **SQLAlchemy** สำหรับจัดการฐานข้อมูล และ **Jinja2** สำหรับระบบเทมเพลต ความเรียบง่ายและความยืดหยุ่นของ **Flask** ทำให้มันเป็นตัวเลือกยอดนิยมสำหรับการพัฒนาเว็บแอป ตั้งแต่โปรเจกต์ต้นแบบไปจนถึงระบบขนาดใหญ่

FLASK MODULE

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     return "Hello, Flask!"
8
9 if __name__ == '__main__':
10     app.run(debug=True)
```

← → ↻ ⓘ 127.0.0.1:5000

Hello, Flask!

Listing 10.15: Example of Flask

```
> python flask.py
* Serving Flask app 'flsk' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 353-208-880
```


FLASK MODULE

```
1 from flask import Flask, render_template_string
2
3 app = Flask(__name__)
4
5 # HTML template as a string
6 html_template = """
7 <!DOCTYPE html>
8 <html lang="en">
9 <head>
10     <meta charset="UTF-8">
11     <meta name="viewport" content="width=device-width, initial-scale=1.0">
12     <title>Greeting Page</title>
13 </head>
14 <body>
15     <h1>Hello, {{ name }}!</h1>
16     <p>Welcome to your simple Flask web app.</p>
17 </body>
18 </html>
19 """
20
21 # Define a route for the homepage
22 @app.route('/')
23 def home():
24     return render_template_string(html_template, name="Alice")
25
26 # Define a dynamic route that takes a name parameter
27 @app.route('/greet/<name>')
28 def greet(name):
29     return render_template_string(html_template, name=name)
30
31 if __name__ == '__main__':
32     app.run(debug=True)
```

Listing 10.16: Example of Flask II



```
> python flaskII.py
* Serving Flask app 'flaskII' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 353-208-880
```


SQLITE MODULE

SQLite เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management System: RDBMS) ที่มีน้ำหนักเบา ไม่ต้องใช้เซิร์ฟเวอร์ และเป็นแบบ self-contained ซึ่งได้รับความนิยมอย่างแพร่หลายในแอปพลิเคชันหลากหลาย เนื่องจากใช้งานง่ายและไม่ต้องมีการกำหนดค่าที่ซับซ้อน

SQLite เป็นโอเพ่นซอร์สและฝังตัวอยู่ภายในแอปพลิเคชันโดยตรง แตกต่างจากฐานข้อมูลแบบดั้งเดิมอย่าง MySQL หรือ PostgreSQL ที่ต้องพึ่งพาเซิร์ฟเวอร์ภายนอก SQLite ทำงานภายในกระบวนการของแอปพลิเคชันโดยตรง ทำให้เหมาะสำหรับโครงการขนาดเล็ก ระบบฝังตัว (embedded systems) อุปกรณ์พกพา และแอปพลิเคชันที่ไม่ต้องการความซับซ้อนของระบบฐานข้อมูลแบบเต็มรูปแบบ

SQLITE MODULE

```
1 import sqlite3
2
3 # Connect to SQLite (creates a database file if it doesn't exist)
4 conn = sqlite3.connect('mydatabase.db')
5
6 # Create a cursor object to interact with the database
7 cur = conn.cursor()
8
9 # Create a table
10 cur.execute('''
11     CREATE TABLE IF NOT EXISTS users (
12         id INTEGER PRIMARY KEY AUTOINCREMENT,
13         name TEXT NOT NULL,
14         age INTEGER NOT NULL,
15         city TEXT NOT NULL
16     )
17 ''')
18
19 # Insert some data into the table
20 cur.execute("INSERT INTO users (name, age, city) VALUES ('Alice', 25, 'New York')")
21 cur.execute("INSERT INTO users (name, age, city) VALUES ('Bob', 30, 'Los Angeles')")
22 cur.execute("INSERT INTO users (name, age, city) VALUES ('Charlie', 35, 'Chicago')")
23
24 # Commit the changes
25 conn.commit()
26
27 # Query the data and display the results
28 cur.execute("SELECT * FROM users WHERE age > 28")
29 rows = cur.fetchall()
30
31 print("Users older than 28:")
32 for row in rows:
33     print(f"ID: {row[0]}, Name: {row[1]}, Age: {row[2]}, City: {row[3]}")
34
35 # Close the connection
36 conn.close()
```

Listing 10.17: Example of SQLite

```
> python Sqlite.py
Users older than 28:
ID: 2, Name: Bob, Age: 30, City: Los Angeles
ID: 3, Name: Charlie, Age: 35, City: Chicago
```

Lecture10 > mydatabase.db

Filter 2 tables..	Rows: 3
Tables	
sqlite_sequence	
users	

	id	name	age	city
1	1	Alice	25	New York
2	2	Bob	30	Los Angeles
3	3	Charlie	35	Chicago



SQLite Viewer v0.5.13

Florian Klampfer | 1,287,821 | ★★★★★ (52)

SQLite Viewer for VSCode

Disable Uninstall Switch to Pre-Release Version ☒ Auto Update 

[DETAILS](#) [FEATURES](#) [CHANGELOG](#)

THANKS

2PM - 3PM

